CS543 WEB SERVICES — ASSIGNMENT 5

Smart Contact Manager (SCM)

HTTP REST resources and headers — Java Spring Boot implementation

Team ID	12
Team members	Aman Bali — 20251651015 Bhupesh Kumar — 20251651032
Technology	Java 21 · Spring Boot 3.2.5 · Maven · JUnit 5 · MockMvc
Service port	8090 (existing SCM Contact Service: 8082)

This report accompanies the source folder, OpenAPI contract, tests and NOTES.md. The Java implementation extends the SCM project; no Python/Flask service is used.

CS543 Assignment 5 — HTTP Methods & Headers for SCM (Spring Boot)

Team ID: 12

Aman Bali — 20251651015

Bhupesh Kumar — 20251651032

Project fit and technology stack

This is the Spring Boot continuation of Assignment 4 for Smart Contact Manager (SCM), not a Python implementation. Stack: **Java 21, Spring Boot 3.2.5, Spring Web, Jakarta Bean Validation, Maven, JUnit 5, Mockito and MockMvc**. The assignment extension runs on port `8090`; it verifies contact references against SCM Contact Service on port `8082` using `CONTACT_SERVICE_URL`.

The uploaded handout says CampusEats. This submission adapts the HTTP-method/header tasks to SCM because SCM is the requested project. Confirm that the instructor accepts the project substitution if the service name is graded strictly.

A1 — SCM method map

Action	Method + URL	Success
Create a share request	`POST /v1/contact-shares`	`201 Created` + `Location`
List/filter/sort/page share requests	`GET /v1/contact-shares?status=pending&sort=created_at&order=desc&limit=20&offset=0`	`200 OK`
Read one share request	`GET /v1/contact-shares/{shareId}`	`200 OK`, or `304 Not Modified` with matching `If-None-Match`
Partially update a pending share request	`PATCH /v1/contact-shares/{shareId}`	`200 OK`, requires current `If-Match`
Delete a share request	`DELETE /v1/contact-shares/{shareId}`	`204 No Content`
Cancel a pending share request	`POST /v1/contact-shares/{shareId}/cancellation`	`200 OK`
Discover collection methods	`OPTIONS /v1/contact-shares`	`204` + `Allow`
Discover item methods	`OPTIONS /v1/contact-shares/{shareId}`	`204` + `Allow`
Discover cancellation methods	`OPTIONS /v1/contact-shares/{shareId}/cancellation`	`204` + `Allow`

Cancellation is a state transition and is modeled as a POST to a sub-resource rather than a verb embedded in the URL.

A3 — Safe and idempotent

Endpoint	Safe?	Idempotent?	Retry behavior
POST create	No	Yes only when the same idempotency key and same payload are reused	Retry with same key; different payload under same key returns 409
GET list	Yes	Yes	Naturally retry-safe
GET one	Yes	Yes	Naturally retry-safe; ETag can yield 304
PATCH update	No	Not inherently	`If-Match` rejects stale updates with 412; read again after an uncertain outcome
DELETE	No	Semantically yes (resource remains absent after deletion)	A repeat returns 404; it cannot delete a second resource

Endpoint	Safe?	Idempotent?	Retry behavior
POST cancellation	No	Not at the response-code level; a repeat returns 409	Re-read state after an uncertain response instead of blindly retrying
OPTIONS	Yes	Yes	No state change

GET does not mutate share records. Create would ordinarily create duplicate records on a retry, so `Idempotency-Key` stores the request fingerprint and original resource. Cancellation is intentionally a conflict on repeat because only a pending request can transition to cancelled.

A5 — OPTIONS and method override

• `OPTIONS /v1/contact-shares` returns `Allow: GET, POST, OPTIONS`.

• `OPTIONS /v1/contact-shares/{shareId}` returns `Allow: GET, PATCH, DELETE, OPTIONS`.

• `OPTIONS /v1/contact-shares/{shareId}/cancellation` returns `Allow: POST, OPTIONS`.

• CORS headers include allowed origin, methods and request headers.

• When `SCM_ENABLE_METHOD_OVERRIDE=true`, a constrained client may send `POST` with `X-HTTP-Method-Override: PATCH` or `DELETE`. This compatibility fallback is disabled by default; direct PATCH/DELETE remains preferred.

B1–B7 — Header behavior

• JSON bodies use `Content-Type: application/json`. Malformed JSON returns 400; an unsupported `Accept` such as `text/html` returns 406.

• Create returns 201 and `Location` for the created resource; read returns 200; delete returns 204 with no body.

• Protected requests require `Authorization: Bearer scm-assignment5-demo-token` by default. Missing/empty/invalid tokens return 401. This is a header-handling demo, not a production token issuer.

• Single-item GET sets `ETag` and `Cache-Control: private, max-age=30, must-revalidate`.

• Rate-limit headers are per client; the default budget is 100/minute. Exceeding it returns 429, `Retry-After`, `X-RateLimit-Limit`, and `X-RateLimit-Remaining`.

• CORS preflight is answered by OPTIONS. Security headers include `X-Content-Type-Options: nosniff` and `Strict-Transport-Security`; HSTS is useful only when deployed over HTTPS.

• Spring Boot's embedded server gzip compression is enabled for eligible JSON responses of 512 bytes or more when the client advertises gzip. The runtime/proxy supplies standard `Date` and `Server` headers.

D2 — Endpoint/header matrix

Endpoint	Request headers	Important response headers
`POST /v1/contact-shares`	Authorization, Accept, Content-Type, Idempotency-Key; optional X-Client-ID	Location, ETag, Content-Type, Idempotency-Replayed on replay, rate-limit headers
`GET /v1/contact-shares`	Authorization, Accept; optional X-Client-ID, Accept-Encoding	Content-Type, rate-limit headers, optional Content-Encoding: gzip
`GET /v1/contact-shares/{id}`	Authorization, Accept; optional If-None-Match	ETag, Cache-Control; 304 has no body
`PATCH /v1/contact-shares/{id}`	Authorization, Accept, Content-Type, If-Match	New ETag on 200; 412 if stale
`DELETE /v1/contact-shares/{id}`	Authorization, Accept	204 no body; security/CORS/rate-limit headers
`POST /v1/contact-shares/{id}/cancellation`	Authorization, Accept	ETag on success; 409 on state conflict
OPTIONS resources	Origin, Access-Control-Request-Method/Headers from browser	Allow, Access-Control-Allow-Origin/Methods/Headers

C4 — Safe-retry plan

Risky endpoint	Mechanism	Reason
Create share	`Idempotency-Key`	Avoids duplicate records when the first response is lost
Read share	`If-None-Match` / ETag	A 304 avoids retransmitting an unchanged representation
Update share	`If-Match` / ETag	A stale write returns 412 rather than overwriting another editor's changes
Cancel share	Re-read state after ambiguous failure	Cancellation is a state transition; the second attempt returns 409

Full HTTP exchange (illustrative; capture actual output before submission)

Request line

```
POST /v1/contact-shares HTTP/1.1
```

Request headers

```
Host: localhost:8090
Authorization: Bearer scm-assignment5-demo-token
Accept: application/json
Content-Type: application/json
Idempotency-Key: scm-a5-demo-001
X-Client-ID: team12-demo
```

Request body

```
{"contactRef":"cnt-existing-contact-id","recipientEmail":"friend@example.com","message":"Please see this contact."}
```

Response status and headers (values vary by run)

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://localhost:8090/v1/contact-shares/shr-generated-id
ETag: "generated-validator"
X-Content-Type-Options: nosniff
Access-Control-Allow-Origin: *
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
```

Response body

```
{
  "share_id": "shr-generated-id",
  "contact_ref": "cnt-existing-contact-id",
  "recipient_email": "friend@example.com",
  "message": "Please see this contact.",
  "status": "pending",
  "created_at": "generated-timestamp",
  "updated_at": "generated-timestamp",
  "cancelled_at": null
}
```

This is an example of the wire format, not actual captured output. Use `capture-curl-transcript.sh` with a real SCM contact ID to produce live evidence.

Eight answers required by the assignment

1. Three endpoints, status and important response header

• Create: POST `/v1/contact-shares` → 201; `Location` identifies the new resource.

• Read: GET `/v1/contact-shares/{id}` → 200; `ETag` lets clients validate cached data.

• Delete: DELETE `/v1/contact-shares/{id}` → 204; the empty body correctly indicates no representation is returned after deletion.

2. Safe and idempotent endpoints

GET and OPTIONS are safe and idempotent. PATCH is not safe and is not inherently idempotent in this design. A normal POST create is not naturally retry-safe because it allocates a new ID; the same `Idempotency-Key` and same request fingerprint return the original resource. Reusing the key for a different body returns 409.

3. ETag, 304 and 412

A single-item GET returns an ETag generated from the public representation. Sending `If-None-Match` with the same value returns `304 Not Modified` with no body, saving bandwidth. Sending an outdated ETag in `If-Match` to PATCH returns `412 Precondition Failed`, preventing a stale editor from overwriting a newer version.

4. 400 versus 422

• 400: POST create with malformed JSON, such as `{`, or a missing/invalid `Idempotency-Key`. The request cannot be parsed or does not meet the request syntax requirements.

• 422: a valid create body references a contact that the SCM Contact Service confirms does not exist, or the list uses an unsupported `status`. The request is syntactically valid but violates a domain rule.

5. Browser blocked despite a 200 in server logs

The browser's CORS/same-origin enforcement may block JavaScript from reading a response even if the server processed the request and logged 200. `Access-Control-Allow-Origin`, plus the preflight method/header permissions, tells the browser which cross-origin calls are permitted. CORS is not authentication.

6. Cache-Control for cache versus no-store

The item GET uses `private, max-age=30, must-revalidate`: the user-specific response may be held in a private client cache briefly, then revalidated. Responses containing credentials or sensitive account data should use `Cache-Control: no-store`; this API never includes the bearer token or password in its representation.

7. When POST is justified for search

GET is appropriate for normal status filtering, sorting and pagination because it is safe, cacheable and shareable. POST may be justified for a very large or sensitive structured search body that does not fit well in a URL. POST sacrifices natural linkability and ordinary GET caching semantics, so it should not be chosen merely because there are several filters.

8. Location on 201 and 3xx

On 201 Created, `Location` identifies the newly created share resource. On a 3xx redirect, `Location` identifies the target URI the client should visit. The same header has a status-dependent meaning.

Run and evidence checklist

```
mvn clean test
mvn spring-boot:run
```

Set `CONTACT_SERVICE_URL=http://localhost:8082/v1/contacts` if required. Use `curl-transcript.txt` only as a checklist; generate `curl-transcript-actual.txt` using the script and replace the placeholder contact ID with one that exists. Do not submit invented terminal output. The rate limiter and share store are in-memory, suitable for the assignment but not a horizontally scaled production deployment.